n = 83 known-log reference row: seeded scalar orbits at K600 and at the cost-model optimum, end-to-end cold

2026-10-19

n = 83 known-log reference row: seeded scalar orbits at K600 and K220, end-to-end cold

Status (2026-10-19): Tier 1 run complete and valid; reading "measured above rho". The protocol and the schedule were committed before any Tier 1 timing, in commit `c04026fa6`. The Tier 1 run is 39307634508, archived in `run_github_39307634508/`. Every number below comes from one of three places, and each is labelled:

- the archived run (39307634508, Tier 1), for **measurements**;
- the cost model, which gives **predictions**;
- the cloud-agent VM, which gives **local diagnostics** (`smoke/`).

Exactness results are exact wherever they were run.

> **This is a separately labelled REFERENCE ROW. It is not the index-calculus baseline, and it is not an IC improvement.** The owner's ruling (2026-10-19) was "Measure both, decide later". This experiment measures the known-log construction beside the IC baseline and does not replace the n = 83 baseline row. Whether this row may ever hold the IC row will be decided after the L1 multiplier-closure result.
>
> **Why it can never beat rho.** A base whose logs are all known needs no relations. Without relations, the method reduces to generic baby-step giant-step on the Frobenius quotient:
>
> - the index of pairs of base points is the baby-step table;
> - the target scan is the giant steps.
>
> It is therefore bounded by the DKM non-adaptive line (crypto `research/lit2026_w3_ggm_audit_20261011/README.md:177-183`), and it can never beat rho. This measurement prices that limit on the baseline's own code. It says nothing about relation-based index calculus.

Result (Tier 1, run 39307634508)

The run completed with exit 0, executed all 209 scheduled rows, and skipped none under the time guard. `verify_receipt.py`, re-run on the archive, reproduces `result.json` and `result_table.md` byte for byte. All 16 gates and all 3 controls pass, so the run is valid. The frozen reading rule gives **measured above rho**.

Primary (pre-registered), 40 cold points, 95% paired bootstrap:

Quantity	Estimate [95%]	Reading
G = E2E(cL220) / cold(rT770)	**2.1919 [1.8584, 2.5893]**	**above rho** (lower bound above 1)
end-to-end cold S of cL220 (this run's rT770 step, 29,512.9 ms per r^(1/2))	0.2303 [0.2059, 0.2577]	

This is not an IC improvement and does not replace the n = 83 baseline row. The reading does not contradict the DKM bound: the known-log base is generic BSGS on the Frobenius quotient and sits above rho, as the bound requires. The "below rho" case, which would have signalled an accounting error, did not occur.

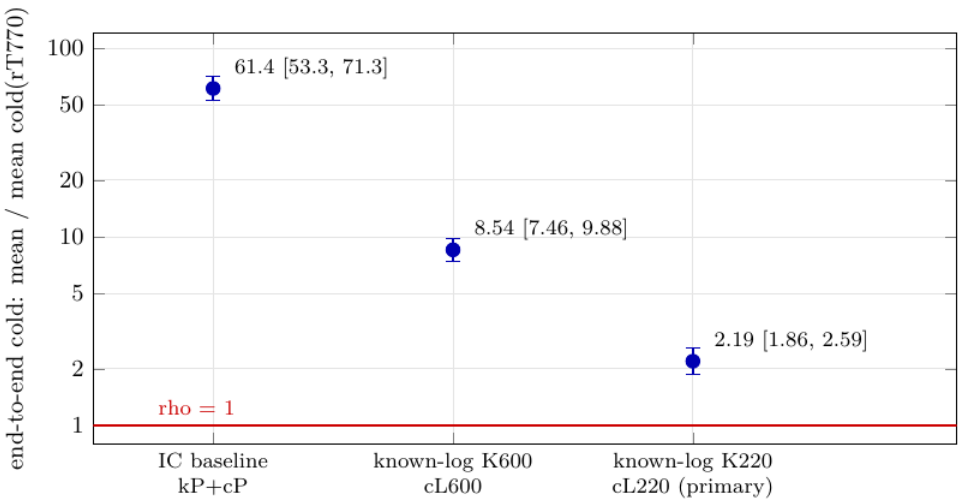


Figure 1: Result. End-to-end cold gap to the tuned rho rT770 for the IC baseline (kP+cP), the known-log base at K600 and the primary cL220; REFERENCE ROW, not an IC improvement. Source: figures/result.tex, vector: figures/result.pdf.

Secondaries (reference, not an IC improvement):

Quantity	Estimate [95%]
E2E(kP+cP) / E2E(cL220)	27.9894 [24.7842, 31.5336]
E2E(kP+cP) / E2E(cL600)	7.1801 [6.8893, 7.4867]

Means: kP+cP 190,268 ms (rank stage 163,779 ms + cP cold 26,490 ms), cL600 26,500 ms, cL220 6,798 ms. The baseline pair was run by the rank-probe experiment's `arm.py` unchanged, and its cP charges no base construction. The known-log arms charge generation.

Descriptive.

Quantity	Estimate [95%]
E2E cold gap kP+cP / rT	61.35 [53.26, 71.28]
E2E cold gap cL600 / rT	8.5446 [7.4573, 9.8800]
E2E cL600 / cL220	3.8982 [3.4860, 4.3638]
online gap cL220 / rT	1.1235 [0.8527, 1.4457]
online gap cL600 / rT	0.1410 [0.0951, 0.1951]
online gap cP / rT	0.1775 [0.1200, 0.2504]
cold cP / cL600	0.9996 [0.9894, 1.0109]

End-to-end cold S: kP+cP 6.447, cL600 0.8979, cL220 0.2303. Online S: cP 0.01722, cL600 0.01368, cL220 0.10898.

Phases (means, ms):

Arm	gen	base load	index	scan self-test	log verify	online	E2E
cL220	78.8	285.4	3,074.6	77.9	33.3	3,216.2	6,797.9
cL600	177.3	315.4	23,228.3	2,213.8	91.6	403.7	26,499.6

ns per online probe: cL220 258.4, cL600 258.3, cP 260.7 (the model assumed 221.4).

Prediction against measurement. Model A predicted E2E 6.04 s at K220 and 22.6 s at K600, and a gap of 2.33 at K220. Model B predicted 5.15 s at K220. The measurement is 6.80 s and 26.5 s, and G = 2.19. K220 is slower than both models in absolute time (the per-probe cost is 258 ns against the assumed 221 ns, and the index build is 3.07 s against about 2.9 s). rT770's mean cold time in this run is 3.10 s against the archived 2.59 s, which brings the gap down to 2.19, near model A's 2.33.

Gates (all pass).

- Every scheduled row valid and present; all 40 points and all 4 rank blocks complete; the receipt matches the schedule; zero rows skipped by the time guard; no contention flags.
- Independent verification of every base log (`check_knownlog.py`), and the verifier detects its three planted corruptions.
- Every known-log process ran the verified files.
- One scalar per point across every IC process and rT, replayed in the oracle; one relation and one probe count per point within each base.
- Q0 matches the archived and frozen identities.
- The producer refuses the planted table (cLX).
- The rank tables equal the archived ones.
- Probes inside the law band (mean over the law: cL220 0.968, cL600 0.905, cP 1.128; maximum over the law 3.80, 3.51, 4.72, all below 20).

Controls (all pass).

- A/A: E2E cL220 / cLaa = 1.0005 [0.9954, 1.0067], inside [0.97, 1.03].
- Planted: the producer refuses the off-by-one table, and the verifier detects all three of its planted corruptions.

What this does and does not say. Without relations, the known-log base is BSGS on the Frobenius quotient, and 2.19 times rho is the price of that construction on the baseline's own code at its best orbit count. It is about 28 times cheaper end to end than the IC baseline in this session only because the baseline pays a 164 s rank stage to find the logs that this construction gets by construction. The ratio says nothing about relation-based index calculus, and the owner's decision on whether this row may ever hold the IC row is pending the L1 multiplier-closure result (`claude/n83-closure`).

Question

Survey lead L2 proposes a different factor base for the n = 83 a = 1 Koblitz curve (related: `IDEA-20261015-7a3c9e`, which identifies folded Koblitz IC with BSGS on the quotient). Instead of B's base, seed the factor base with 600 scalar orbits [a_j]G:

- their logs a_j are known by construction;
- so there is no rank stage and no linear algebra (`LAnone`);
- the rank-probe producer already reads such a base as a retained header, and its log table through `KIC_LOG_TABLE_IN`.

The same construction was used before, at n = 53 (complete) and as an n = 83 screen: `experiments/koblitz-pair-claw-20260929/README.md:339-400`.

What does this cost **end to end, cold**, on new public points? End to end means generating the base and log table, loading the base, building the index, verifying the logs, the self-tests and the one-target online interval. The measurement is made at two orbit counts:

- K600, the baseline's orbit count;
- K_opt, the cost model's optimum.

For each, how far is it from the tuned rho rT770, and from the IC baseline (kP+cP) measured in the same session?

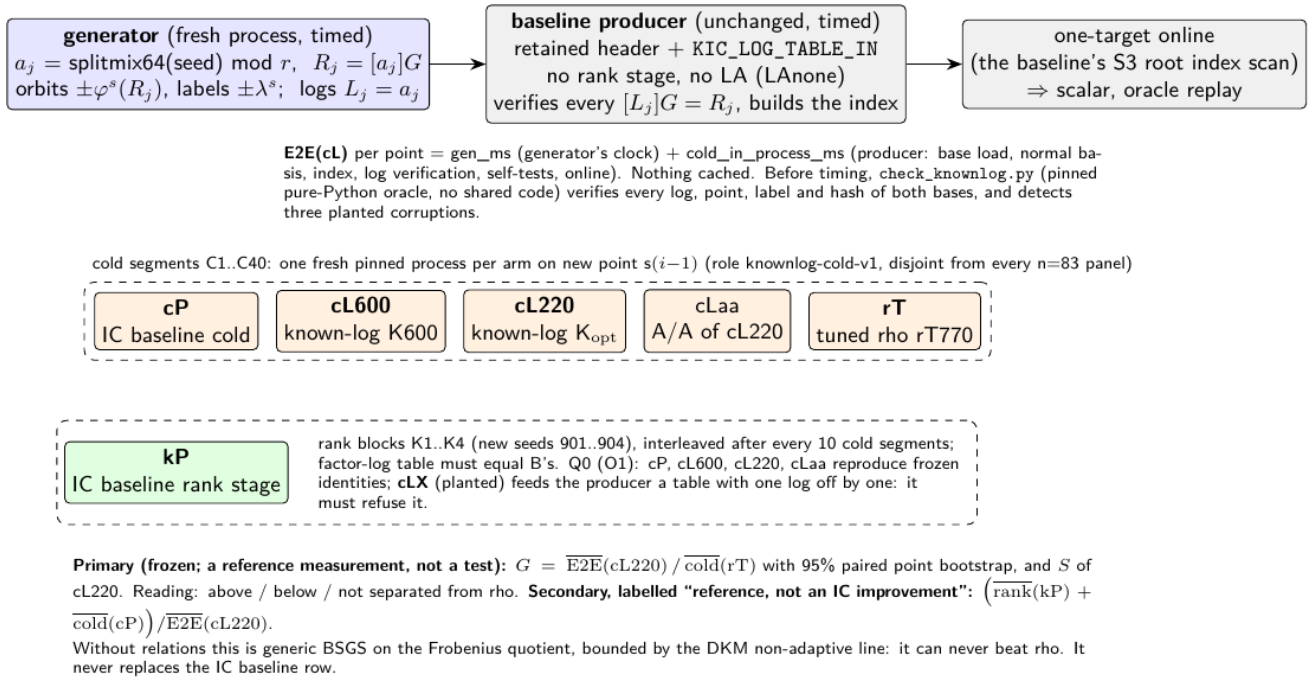


Figure 2: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

The construction (zero producer code)

Generator. `knownlog/` is a Rust crate with no dependencies. It writes out GF(2^83), the curve and SHA-256 itself, so it shares no code with the producer that reads its output. It produces:

- **Scalars.** a_j is the j-th output of splitmix64(seed = 20261019), rejection-sampled below floor(2^64/r)·r, reduced mod r, with 0 rejected. The K220 base is therefore the first 220 orbits of the K600 base.
- **Representatives.** R_j = [a_j]G, computed by a fixed-base comb over [2^i]G with one batched inversion per comb row.
- **Base points.** These follow B's header layout, which was checked against B's K600 header point by point. For column j and s = 0..82, the points are φ^s(R_j) with label [j, λ^s mod r], then $-\varphi$^s(R_j) with label [j, $-\lambda$^s mod r]. That gives 166·K points and K signed-Frobenius columns. The generator asserts that the orbits are distinct.
- **Factor-log table.** logs[j] = a_j.

The generator's own clock runs from the start of `main` until both files are written and synced.

Producer. The baseline's rank-probe producer (digest `d4394df3...`) runs unchanged. It reads the header as a retained base. It checks the table against the base's hash and verifies every [L_j]G = R_j before any target work; that check is charged to setup. It then builds the S3 root index over the folded state vector: $83K^2$ logical states and $(83K^2 + K)/2$ stored states. Finally it solves the one public point with the baseline's online scan (lift_scan, Feistel order, prefetch, 16-byte compact table).

Charge. Each known-log process starts cold, with nothing cached. Its cost is E2E = gen_ms + the producer's `cold_in_process_ms`. Every phase is charged, including base and table generation. The IC baseline arm cP does not charge B's base construction (X ≈ 0.43 s, as in the baseline row).

Exactness. Every scalar is replayed in the pinned oracle. Before any timing, `check_knownlog.py` verifies the logs and the whole base independently, using the pinned pure-Python oracle of crypto 770f6a69 and `hashlib`. The independent verifier checks:

- every [a_j]G = R_j;

- the scalars against the declared rule;
- every base point as $\pm\varphi\hat{}s(R_j)$ with label $\pm\lambda\hat{}s$;
- that the orbits are distinct;
- the base hash and the frozen file digests.

The log of every base point, the sum of coefficient·L_column, is therefore known and verified.

Choice of K (prediction; `cost_model.py`, `analysis/cost_model.json`)

With no rank stage, the end-to-end cold cost is the setup plus the online interval. The terms scale as follows:

- **Index build and Feistel self-test.** Both scale with the $83K^2$ logical states. At K600 they take 19,667 ms and 1,882 ms (archived cP means, run 39214027289).
- **Online interval.** This is P(K)·ns_probe, with the archived probe law $P(K) = r / (166 \cdot 83 \cdot K^2)$:
  - the 2,400 kP rank attempts of that run average 0.9785 of the law at K600, with a coefficient of variation of 0.97, close to exponential;
  - ns_probe is 221.4 ns, cP's online cost per probe.
- **Linear terms.** Base load and log verification scale with K.
- **Generation** is linear in K (local).

The model has two variants:

- **Model A** uses the pure scaling above. Its minimum is at $K^* = 217$, at 6.04 s.
- **Model B** also lets the per-state index cost and the per-probe cost fall with table size, with the exponents of the local K220/K600 smoke (0.094 and 0.257). Its minimum is at $K^* = 198$, at 5.05 s.

The frozen **K_opt = 220** is the even multiple of 10 nearest model A's minimum. It is within 0.04% of model A's minimum and within 2.0% of model B's.

	model A	model B
predicted E2E at K220 (s)	6.04	5.15
of which index + self-test / online (s)	2.90 / 2.85	2.66 / 2.20
predicted gap to rT770 at K220 (2,594 ms archived)	2.33	1.99
predicted E2E at K600 (s), gap	22.6, 8.7	22.6, 8.7

This agrees with the lead's own estimate: about 5.5 s at $K \approx 210$ and about 22 s at K600, against 166.0 s for the baseline end to end.

One known bias. Locally, base load is mostly a fixed cost: 382 ms at K220 against 463 ms at K600. Model A's linear base-load term therefore underestimates K220 by about 0.2–0.3 s. A fixed cost does not move the minimum.

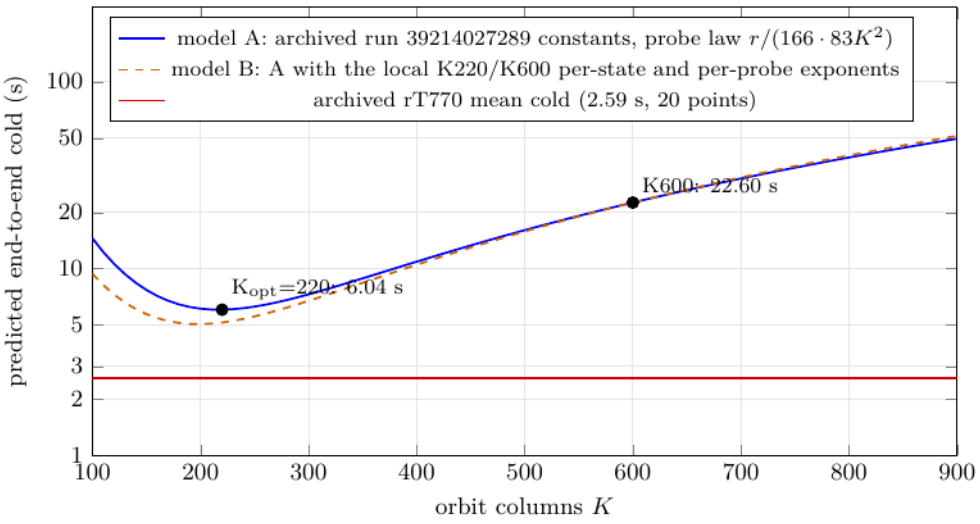


Figure 3: Predicted end-to-end cold cost against K (prediction, not a measurement; source figures/cost_model.tex from `make_cost_figure.py`, vector figures/cost_model.pdf).

Frozen Tier 1 design

The run uses one GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, and one fresh process per run. The baseline arms and the rho arm run through `../koblitz-n83-rankprobe-20261019/arm.py` **unchanged**. The known-log arms run through this directory's `arm.py`.

Arm	Kind	Role
`cP`	cold, B's base + B's cached table	**IC baseline** cold arm, in this session for pairing
`kP`	rank stage (`KIC_RANK_SOLVE=frame`)	**IC baseline** rank stage; its table must equal B's
`cL600`	known-log, K = 600	same orbit count as the baseline; generation charged
`cL220`	known-log, K = K_opt = 220	**primary**
`cLaa`	known-log, K = 220	A/A of cL220, on every point
`cLX`	known-log K = 220, planted (Q0 only)	the table with log column 1 off by one; the producer must refuse it
`rT`	tuned rho rT770 (Fold/64/DP6)	the reference rho, seed 990000 + point seed

Panel. 40 new public points, `knownlog-cold-v1` (`inputs/cold_targets.json`, built by `make_targets.py`). They are regenerated and checked disjoint from every n = 83 panel of the law:

- Q0 and B's panel;
- scanorder-heldout, combined-primary-v1, probepipe-primary-v1, statefold-cold-v1, tablefilter-primary-v1, stacked-cold-v1 and rankcut-cold-v1;
- the 64 `eval` points of n131-ref-walk-20261015, and rankprobe-cold-v1;
- the roles registered on the n83 branches and scratch worktrees: indexlift-cold-v1, rotkey-cold-v1, onlineframe-v1, k899-cold-v1 and closure-cold-v1.

No scalar is known for any of the points.

Order.

- O1 (Q0): cP, cL600, cL220, cLaa, cLX.

- C1–C40: one cold point each, with cP, cL600, cL220, cLaa and rT shuffled.
- K1–K4: one kP process each, with the new rank seeds 901–904 on units s000–s003, interleaved after every ten cold segments.

That is 209 rows. The time guard is 200 min, and the job limit is 260 min. The estimate is about 1.2 h including the build.

Primary (pre-registered): the reference measurement itself.

- G = mean E2E(cL220) / mean cold(rT) over the 40 points. The interval is a 95% bootstrap: points resampled, paired, 10,000 draws, seed 20261019.
- Alongside it, the end-to-end cold `S` of cL220 = mean E2E / (this run's rT770 ms per step · $\sqrt{r}$), with its interval.
- **Reading rule (frozen):**
  - "measured above rho" iff the lower bound of G exceeds 1;
  - "measured below rho" iff its upper bound is below 1. This would contradict the DKM bound for a generic method, so the accounting would be audited before any claim;
  - otherwise "not separated";
  - **invalid** if any gate or control fails or any scheduled row is missing.
- **No outcome is an IC improvement**, and none replaces the baseline row.

Secondary, labelled "reference, not an IC improvement".

- E2E(kP+cP) / E2E(cL220): mean kP rank stage + mean cP cold, over mean cL220 E2E. The bootstrap resamples blocks, then attempts within each drawn block, and points, paired.
- The same ratio with cL600 as the denominator.
- Descriptive only:
  - the gaps to rT of kP+cP and cL600;
  - cL600 / cL220;
  - the online gaps;
  - phases per arm, ns per probe, the probe-law ratios;
  - the model's prediction beside the measurement.

Gates.

- **Every base point's log is verified, independently.** Before timing, the runner-built generator writes both bases. `check_knownlog.py` (the pinned oracle, sharing no code with the generator or the producer) must PASS on both, with the frozen file digests, and must detect its three planted corruptions. In the timed processes:
  - every cL process's freshly generated files must hash to those verified files;
  - the producer must verify all K logs in process;
  - the state counts, slot count, root-table entries and index digests must equal the frozen ones.
- **One scalar per point** across every IC process (cP, kP, cL600, cL220, cLaa), equal to rT's, replayed as [d]G = Q in the pinned oracle.
- **One relation and one probe count per point within each base**: cP with kP; cL220 with cLaa.
- **Q0** identities:
  - cP: the archived Feistel identity;
  - cL600, cL220 and cLaa: the frozen identities of their bases.
- **kP** must reach full rank, with a written table equal to B's.
- **Probes inside the law band.** For cP, cL600 and cL220, the mean over the 40 points of probes / (r / (166·83·K²)) must lie inside [0.5, 1.5], and no point may exceed 20.

Controls.

- **A/A:** E2E cL220 / cLaa must lie inside [0.97, 1.03].
- **Planted:** cLX must be refused by the producer, and the verifier's three planted corruptions must be detected.

Power (`power_check.py`, `analysis/power.json`; 400 simulations per cell, 1,000 bootstrap draws each). The priors are:

- the archived rT770 cold times (20 points, coefficient of variation 0.43);
- the model's setup at K220, with the archived cP setup spread (0.36%);
- exponential online probes at 0.9785 of the law.

The rule is the smallest N in {20, 30, 40, 60} that has, under both models, a median relative half-width of the 95% interval of at most 0.20 and P(lower bound > 1) ≥ 0.90. It chose $\mathbf{N = 40}$:

N = 40	true gap	median relative half-width	P(lower bound > 1)	coverage	secondary kP+cP / cL220 (median)
model A	2.30	0.186	1.00	0.93	27.7
model B	1.97	0.181	1.00	0.95	32.6

At N = 20 and N = 30 the half-width is 0.25 and 0.21, so N = 40 is the first grid value at or below 0.20. Percentile coverage is 0.90–0.95, slightly below the nominal 0.95, because the online cost is skewed.

Local smoke (cloud-agent VM; diagnostic)

See `smoke/README.md`. In short:

- The generator's tests pass, and its output is deterministic.
- The independent verifier passes both bases and detects all three planted corruptions.
- On Q0, all five O1 rows are valid. cP reproduces the archived identity. cL600, cL220 and cLaa recover 3141592653589793.
- The producer refuses the planted table.
- Local times on Q0: cL220 4.47 s end to end, cL600 31.1 s, cP 31.4 s.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`knownlog/`	the known-log base generator (Rust, no dependencies) and its tests
`check_knownlog.py`	independent verification of a base and its log table in the pinned oracle, with planted corruptions
`arm.py`, `run_schedule.py`	the known-log arms (generator + the baseline producer); the schedule runner (baseline and rho arms through the rank-probe experiment's `arm.py`)
`verify_receipt.py`	gates, statistics, the frozen reading rule; `result.json`, `result_table.md`
`restore.py`	restores crypto 770f6a69 and derives the baseline producer through `../koblitz-n83-rankprobe-20261019/restore.py` (its own pins), and checks this directory's pins
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`make_power_prior.py`, `inputs/power_prior.json`	the archived constants (run 39214027289)
`cost_model.py`, `analysis/cost_model.json`, `make_cost_figure.py`, `figures/cost_model.*`	the K choice (prediction) and its graph
`power_check.py`, `analysis/power.json`	precision of the primary, the panel size
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`run_github_39307634508/`, `figures/make_result_figure.py`, `figures/result.*`	the archived Tier 1 run (`panel/result.json`, `result_table.md`), and the result figure
`smoke/`	local diagnostics
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`